

LLM FUNDAMENTALS

AI Basics — Complete Revision Notes

By codestorywithMIK



Education in MIK's world is free. Always has been, always will be. 🧐

If my AI playlist or channel has helped you in any way, and you'd like to buy me a coffee ☕ — feel free, it's always appreciated (never expected) ❤️

☕ **UPI: imammazhar4@ybl**

Contents

01	What is an LLM?
02	Tokens — the puzzle pieces of text
03	Embeddings — turning words into numbers
04	Vector Databases — AI's filing system
05	RAG (Retrieval-Augmented Generation)
06	Transformers & Attention
07	Q, K, V — Query, Key, Value
08	Why Self-Attention Is Slow (and how GPUs fix it)
09	The Context Window
10	The "Lost in the Middle" Effect
11	Parameters — the model's dials
12	Pre-Training
13	Fine-Tuning
14	Inference, Temperature & Hallucinations

Plus: a one-page master flowchart, a RAG-vs-Fine-Tuning interview cheat sheet, and a short glossary at the very end.

01 What is an LLM?

The world's most well-read autocomplete.



IN SIMPLE WORDS

- LLM = Large Language Model.
- **Large** — trained on a huge slice of text from the internet, books, and articles.
- **Language** — it has learned statistical patterns in human writing.

LLMs are not magic brains. Their core job is simple: look at the text so far and predict the most likely next token. Because they have seen an enormous amount of writing during training, their guesses are good enough to feel like real conversation — but underneath, it is still prediction, not understanding in the human sense.

Key takeaway — An LLM doesn't "know" things the way a person does — it predicts plausible continuations of text based on patterns learned from its training data.

02 Tokens

Small puzzle pieces of text that the model actually reads.



IN SIMPLE WORDS

- A token is a chunk of text — sometimes a whole word, sometimes part of one, sometimes just a punctuation mark or space.
- Common short words are usually a single token; longer or rarer words get split into pieces.

Before an LLM can do any math, it has to chop your sentence into tokens and convert each one into a number. "I love playing!" might become five tokens, as shown below.

I	love	play	ing	!
---	------	------	-----	---

Key takeaway — More tokens mean more compute — most AI APIs bill by the token, so verbose prompts and long documents cost more, not just take longer.

03 Embeddings

Translating words into numbers so their meaning can be compared.



IN SIMPLE WORDS

- An embedding is a list of numbers (a vector) that represents a word's or sentence's meaning.
- Think of a vector as a coordinate: it places the word at a specific point in a high-dimensional "meaning space."

Words with similar meaning end up with similar vectors. To measure "similar," models commonly use cosine similarity or a dot product between two vectors — a higher score means the vectors point in a more similar direction, i.e. more related meaning.

	Pet-like	Has tail	Can fly
Dog ■	0.9	0.9	0.0
Airplane ✈️■	0.0	0.0	1.0

Key takeaway — Vectors that point in similar directions represent similar meanings; vectors that point in very different directions represent unrelated concepts.

04 Vector Databases

The filing system where AI stores and searches embeddings.



IN SIMPLE WORDS

- Semantic search means searching by meaning, not by exact spelling or keywords.

A vector database is built to store and search millions of embeddings efficiently. Instead of exact keyword matching, it finds vectors that are mathematically close together — so a query about "remote work options" can retrieve a document titled "Work From Home Policy" even though no words match exactly.

1. User asks	Can you provide remote work options?
2. Search	Look for stored vectors close in meaning to the query
3. Match found	A chunk titled "Work From Home Policy"

Key takeaway — Exact words don't need to match for a vector search to work — only the underlying meaning needs to be close.

05 RAG (Retrieval-Augmented Generation)

Giving an LLM access to your private or current data without retraining it.



IN SIMPLE WORDS

- Chunking — splitting a large document into small, searchable paragraphs before storing them.
- RAG — finding the most relevant chunk(s) and inserting them into the prompt so the model can "read" them before answering.

You don't need to retrain a model to teach it new facts. Instead, you store your documents as embeddings, retrieve the most relevant pieces for a given question, and paste them into the prompt right before the model generates its answer.

User prompt	→	Vector DB search	→	Retrieve best chunk(s)	→	Answer, grounded in that text
-------------	---	------------------	---	------------------------	---	-------------------------------

Key takeaway — RAG hands the model an open book right before the exam — it's fast to set up, cheap to update, and ideal for private or frequently-changing information.

06 Transformers & Attention

The architecture that lets AI understand the context of a sentence.



IN SIMPLE WORDS

- Transformer — the overall neural network architecture (think: the car).
- Attention — the core mechanism inside it that relates words to each other (think: the engine).

Using self-attention, the model looks at every other word in the sentence and scores how relevant each one is to the word it's currently processing. In *"The boy went to the shop because he needed milk,"* attention lets the model learn that "he" refers to "the boy," not "the shop" or "the milk," based on patterns learned during training — not hard-coded grammar rules.

Key takeaway — Self-attention is how transformers track which words relate to which other words, across an entire sequence, without needing to process it strictly left-to-right.

07 Q, K, V — Query, Key, Value

How attention mathematically matches what you want with what's available.



IN SIMPLE WORDS

- Softmax converts raw attention scores into percentages that add up to 100%.

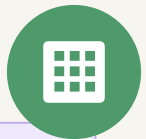
For every token, the model computes a Query vector (what it's looking for), and compares it against every other token's Key vector (an identifier used to judge relevance). The result is a set of raw attention scores, which softmax turns into weights — for example, the token "cat" might receive 80% of the attention while "is" receives only 5%. Those weights are then used to combine the Value vectors — the actual content — into the final output.

Query (Q)	Key (K)	Value (V)
What we're looking for	The labels checked for relevance	The actual information retrieved

Key takeaway — Query asks the question, Key advertises relevance, and Value carries the information that actually gets blended into the output.

08 Why Self-Attention Is Slow

The N^2 problem: every token needs to "talk" to every other token.



IN SIMPLE WORDS

- GPUs (Graphics Processing Units) contain thousands of small cores built to run matrix math in parallel, which is exactly what attention needs.

Self-attention's cost grows quadratically with sequence length, which is why very long contexts are expensive to process. Specialized hardware and algorithmic tricks are what make modern long-context models practical.

Problem	Why it's expensive
The N^2 problem	Every token compares itself with every other token — 1,000 tokens means roughly 1,000,000 pairwise comparisons.
The fix	GPUs (and TPUs) run these comparisons as massive parallel matrix multiplications instead of one at a time; techniques like FlashAttention reduce memory overhead further.

Key takeaway — Attention cost scales roughly with the square of the number of tokens — this is a major reason longer context windows cost more compute, not just more memory.

09 The Context Window

The model's working memory limit for a single conversation.



IN SIMPLE WORDS

- Context = the current conversation (short-term, resets every chat).
- Memory (training knowledge) = what the model learned during pre-training (long-term, baked into its parameters).

The context window holds everything the model can "see" at once: system instructions, pasted documents, prior messages, and the current prompt — commonly measured in tokens (e.g. 128k, 200k, 1M). If a conversation exceeds that limit, the oldest content typically gets dropped or summarized, depending on the application.

Key takeaway — A bigger context window lets you paste in more material, but it doesn't automatically make the model reason better — it just gives it a bigger desk to work on.

10 The "Lost in the Middle" Effect

Models tend to recall the beginning and end of long inputs better than the middle.



Research on long-context recall has repeatedly found a U-shaped accuracy curve: information placed at the very start or very end of a long prompt is retrieved more reliably than information buried in the middle.

Position in context	Typical recall accuracy
Beginning	High
Middle	Noticeably lower
End	High

Key takeaway — Put your most important instructions or facts at the very beginning or the very end of a long prompt — don't bury critical details in the middle of a large document dump.

11 Parameters

The billions of adjustable numbers ("dials") a neural network tunes to learn patterns.



IN SIMPLE WORDS

- Weights and biases are the specific numerical values inside a neural network — informally, its "dials."
- "70B" means 70 billion parameters, not 70 billion memorized facts.

During training, the model makes a guess, compares it to the correct answer, and nudges its parameters slightly so the next guess is a little closer. Repeated billions of times across a huge dataset, this process is what tunes a model's parameters to the exact configuration needed to predict text accurately.

Key takeaway — More parameters generally mean more capacity to capture complex patterns — but a larger parameter count is not itself a store of facts; it's a store of learned statistical relationships.

12 Pre-Training

Teaching a model the basic patterns of language by having it read a huge amount of text.



In pre-training, a model is fed enormous amounts of text and repeatedly trained on simple self-supervised tasks — mainly predicting the next token, or filling in masked-out words. This produces a **base model**: it knows how language works and has absorbed a lot of general knowledge, but it is not yet tuned to reliably follow instructions or hold a conversation.

Task	Example
Predict next token	"The cat drank the ____" → "milk"
Fill in the blank	"An ____ a day keeps the ____ away" → "apple" / "doctor"

Key takeaway — A freshly pre-trained base model can complete text fluently, but it isn't yet an obedient assistant — that comes from the next stage, fine-tuning.

13 Fine-Tuning

Giving a pre-trained model a specialized education to turn it into an expert or a reliable assistant.



IN SIMPLE WORDS

- You cannot fine-tune a model just by typing "act like a doctor" in a prompt — that's prompting, not fine-tuning.

Fine-tuning takes the pre-trained base model and continues training it on a smaller, carefully curated dataset — for example, instruction-following examples, or thousands of annotated clinical notes. This further adjusts the model's parameters, shaping its tone, behavior, or domain expertise in a way that persists across every future conversation, not just the current prompt.

Key takeaway — Fine-tuning changes the model itself (permanently, until retrained again); prompting only changes what you show it for the current conversation.

Interview prep: RAG vs. Fine-Tuning

Dimension	RAG	Fine-Tuning
Analogy	An open-book exam	Years of study to become a specialist
What it changes	Adds temporary external context at query time	Permanently updates the model's internal parameters
Best for	Private company data, frequently-changing facts, real-time info	Changing tone/style, or teaching a durable new skill (e.g. coding conventions, domain diagnosis)
Cost & speed	Fast and cheap to set up	Slower, more expensive, needs quality training data

14 Inference, Temperature & Hallucinations

How predictable or creative an answer sounds, and why models sometimes confidently make things up.



IN SIMPLE WORDS

- Inference = using an already-trained model to generate an answer (as opposed to training, which updates the model).

The model doesn't pick its next word randomly; it scores every candidate token and samples from that distribution. **Temperature** controls how sharply skewed that sampling is — low temperature strongly favors the single most likely word, high temperature spreads probability across more options, producing more varied but less predictable text.

Hallucinations happen because a model has no built-in "I don't know" reflex — it is optimized to produce a plausible-sounding continuation even when it lacks the relevant facts (for instance, about events after its training cutoff, or about a topic it was rarely exposed to). It will stitch together a fluent, confident-sounding answer that can still be wrong.

Setting	Behavior	Example continuation of "My favorite food is..."
Low temperature	Picks the highest-probability word — predictable, focused	"...pizza."
High temperature	Allows lower-probability words — more varied, more creative (and more error-prone)	"...the memory of rainy afternoons in my mother's kitchen."

Key takeaway — Never take an LLM's factual claims at face value without verification — grounding techniques like RAG or live web search reduce, but don't eliminate, hallucination risk.

The Master Flow: From Prompt to Output

A single request moves through every concept in this guide, in order:

1. User prompt	"Who is the CEO?" — raw text input.
2. Tokenizer	Chops the prompt into tokens (Ch. 2).
3. Embeddings	Converts tokens into numeric vectors (Ch. 3).
4. (Optional) RAG lookup	A vector DB retrieves relevant company data and adds it to the context (Ch. 4–5).
5. Context window	Holds the full prompt, instructions, and any retrieved data at once (Ch. 9–10).
6. Transformer engine	Self-attention with Q/K/V relates every token to every other token, computed in parallel on GPUs (Ch. 6–8).
7. Parameters	Billions of trained weights process the math (Ch. 11–13).
8. Temperature & sampling	Chooses how predictable vs. varied the output should be (Ch. 14).
9. Output	The generated answer — always worth verifying (Ch. 14).

Glossary

Term	Meaning
Token	The smallest unit of text a model processes — a word, part of a word, or symbol.
Embedding	A numeric vector representing the meaning of a token, phrase, or document.
Vector database	A database optimized to store and search embeddings by semantic similarity.
RAG	Retrieval-Augmented Generation — injecting retrieved external text into a prompt.
Attention	The mechanism that lets a model weigh how relevant each token is to every other token.
Context window	The maximum amount of text (in tokens) a model can consider at once.
Parameters	The learned numerical weights inside a neural network.
Pre-training	Initial large-scale training on general text using self-supervised objectives.
Fine-tuning	Further training a pre-trained model on a smaller, curated dataset.
Temperature	A setting that controls how much randomness is allowed when sampling the next token.
Hallucination	A fluent but factually incorrect model output.

“Always remember — your only real competition is who you were yesterday. Don't chase someone else's pace; chase your own progress. Show up, stay consistent, and let compounding do the rest.”

— MIK

Education in MIK's world is free. Always has been, always will be. 🧐

If my AI playlist or channel has helped you in any way, and you'd like to buy me a coffee ☕ — feel free, it's always appreciated (never expected) ❤️

☕ **UPI: imammazhar4@ybl**